

SQL FUNDAMENTALS

A Complete Guide to SQL Fundamentals

From Basics to Practice

Prepared by Adedamola Philip~Pharez

About This Guide

This guide is written for someone who already works with databases or has an idea practically but wants to understand the theory and terminology behind what they already do every day.

Chapter 1: What is SQL?

SQL stands for Structured Query Language. It is the standard language used to communicate with relational databases. Whether you are fetching a user's profile, creating a table, or deleting old records, SQL is what makes that happen.

1.1 What is a Database?

A database is an organized collection of data stored electronically. Think of it like a highly structured Excel file, but far more powerful, scalable (**Scalable means a system can handle increasing volumes of data, users, or workload without significant performance problems or failures.**), and designed for multiple people and systems to use simultaneously.

1.2 What is a Relational Database?

A relational database stores data in tables rows and columns and allows those tables to relate to each other. Examples include:

- MySQL - very common in web applications
- PostgreSQL - powerful open source database
- Microsoft SQL Server - common in enterprise environments
- SQLite - lightweight, used in mobile apps
- Oracle - used in large-scale enterprise systems

1.3 Key Concepts You Need to Know

Term	What It Means	Real World Example
Table	A collection of related data, like a spreadsheet	A 'users' table storing all user info
Row / Record	A single entry in a table	One user: id=1, name='Sara'
Column / Field	A specific attribute in a table	The 'email' column
Primary Key	A unique identifier for each row	User ID - no two users have the same ID
Foreign Key	A column that links to another table's primary key	order.user_id pointing to users.id
Schema	The structure/blueprint of the database	All the table definitions and relationships
Query	A command you send to the database	SELECT * FROM users
Index	A performance booster for searching data	Index on email column for fast lookup

Chapter 2: The Categories of SQL

SQL is not just one thing, it is organized into different sub-languages, each serving a specific purpose. These are called sublanguages or command categories.

Category	Full Name	What It Does	Key Commands
DDL	Data Definition Language	Defines and manages the structure of the database	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	Works with the actual data inside tables	SELECT, INSERT, UPDATE, DELETE
DCL	Data Control Language	Controls access and permissions	GRANT, REVOKE
TCL	Transaction Control Language	Manages transactions and changes	COMMIT, ROLLBACK, SAVEPOINT

Focus of This Guide

This guide focuses on DDL and DML - the two you use most often as a Data Analyst/Analytics Engineer.

Chapter 3: DDL - Data Definition Language

DDL is everything related to defining and managing the structure of your database. When you write DDL, you are not touching the data inside the tables, you are working on the tables themselves.

Think of it this way

Analogy: DDL is the architect designing and building the house. DML is the person living in it and moving furniture around.

3.1 CREATE - Building a Table

The CREATE command is used to create a new table, database, or other database object. This is what you write when setting up a project for the first time.

```
-- Creating a database
CREATE DATABASE my_app;

-- Creating a table inside it
CREATE TABLE users (
  id          INT          PRIMARY KEY AUTO_INCREMENT,
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(150) UNIQUE NOT NULL,
  age         INT,
  created_at  DATETIME     DEFAULT CURRENT_TIMESTAMP
);
```

Column Constraints

When creating a table, you define rules (constraints) for each column:

Constraint	What It Does
PRIMARY KEY	Uniquely identifies each row. Only one per table.
NOT NULL	The column cannot be empty — a value is required.
UNIQUE	No two rows can have the same value in this column.
DEFAULT	Automatically sets a value if none is provided.
AUTO_INCREMENT	Automatically increases the number for each new row. Syntax varies by database, see table below.
CHECK	Ensures values meet a specific condition (e.g., age > 0).
FOREIGN KEY	Links this column to a primary key in another table.

Auto-Increment Across Databases

The AUTO_INCREMENT constraint works differently depending on which database you use. Here is how each database handles it:

Database	Syntax	Notes
MySQL	AUTO_INCREMENT	Most common. Place after the column type. Starts at 1, increments by 1.
SQL Server	IDENTITY(1,1)	IDENTITY(start, step) — e.g. IDENTITY(100,5) starts at 100, goes up by 5.
PostgreSQL	SERIAL or GENERATED ALWAYS AS IDENTITY	SERIAL is older shorthand. GENERATED ALWAYS AS IDENTITY is the modern standard.
SQLite	AUTOINCREMENT	Optional SQLite auto-assigns rowid without it. Adding AUTOINCREMENT enforces strict uniqueness.
Oracle	GENERATED AS IDENTITY	Available from Oracle 12c onwards.

```
MySQL
id INT PRIMARY KEY AUTO_INCREMENT

SQL Server
id INT PRIMARY KEY IDENTITY(1,1)

PostgreSQL (modern)
id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY

SQLite
id INTEGER PRIMARY KEY AUTOINCREMENT

Oracle
id NUMBER GENERATED AS IDENTITY PRIMARY KEY
```

3.2 ALTER - Changing a Table's Structure

ALTER lets you modify an existing table — adding columns, changing data types, renaming columns, or adding/removing constraints. You use this when your data requirements change after the table already exists.

```
Add a new column
ALTER TABLE users ADD COLUMN phone VARCHAR(20);

Change a column's data type
ALTER TABLE users MODIFY COLUMN age TINYINT;

Rename a column
ALTER TABLE users RENAME COLUMN name TO full_name;

Add a constraint
ALTER TABLE users ADD CONSTRAINT chk_age CHECK (age >= 0);

Drop a column
ALTER TABLE users DROP COLUMN phone;
```

3.3 DROP - Deleting a Table

DROP permanently deletes a table and everything inside it. This cannot be undone. Use it with extreme caution in production environments.

```
Drop a table (deletes everything, structure AND data)
DROP TABLE users;

Drop a database (deletes all tables inside it too)
DROP DATABASE my_app;

Safe version: only drops if the table exists
DROP TABLE IF EXISTS users;
```

Warning

DROP removes both the structure AND the data permanently. TRUNCATE removes only the data but keeps the structure.

3.4 TRUNCATE - Clearing All Data

TRUNCATE removes all rows from a table instantly, but the table itself (its structure, columns, constraints) remains intact. It is faster than DELETE for wiping all rows because it does not log individual row deletions.

```
**Remove all data from users table, keep the table structure
TRUNCATE TABLE users;
```

Command	Removes Data?	Removes Structure?	Can Be Rolled Back?	Speed
DROP	Yes	Yes	No	Fast
TRUNCATE	Yes	No	No (usually)	Very Fast
DELETE (all rows)	Yes	No	Yes	Slower

Chapter 4: DML - Data Manipulation Language

DML is where your application lives. Every time a user logs in, signs up, updates their profile, or you fetch a list of products that is DML. It works with the data inside the tables defined by DDL.

Think of it this way

Analogy: DML is everything that happens inside the house cooking, sleeping, rearranging furniture. The house itself does not change, only what is happening inside.

4.1 SELECT - Reading Data

SELECT is the most used SQL command. It fetches data from one or more tables. You can filter, sort, group, and join data all within a SELECT statement.

```
Get all columns from users table
SELECT * FROM users;
```

```
Get specific columns only
SELECT id, name, email FROM users;
```

```
Filter with WHERE
SELECT * FROM users WHERE age > 18;
```

```
Multiple conditions
SELECT * FROM users WHERE age > 18 AND email LIKE '%@gmail.com';
```

```
Sort results
SELECT * FROM users ORDER BY name ASC;
```

```
Limit results
SELECT * FROM users LIMIT 10;
```

```
Pagination
SELECT * FROM users LIMIT 10 OFFSET 20;  -- page 3
```

WHERE Clause Operators

Operator	Meaning	Example
=	Equals	WHERE age = 25
!= or <>	Not equal	WHERE status != 'inactive'
> / <	Greater / Less than	WHERE age > 18
>= / <=	Greater/Less than or equal	WHERE score >= 50
BETWEEN	Within a range (inclusive)	WHERE age BETWEEN 18 AND 35
LIKE	Pattern matching	WHERE name LIKE 'S%'
IN	Matches any value in a list	WHERE id IN (1, 2, 3)
IS NULL	Value is empty/missing	WHERE phone IS NULL
AND / OR	Combine conditions	WHERE age > 18 AND active = 1

4.2 INSERT - Adding New Data

INSERT adds new rows to a table. When a user signs up, fills a form, or creates a post INSERT is what saves that data.

```
Insert one row (specify columns explicitly-best practice)
INSERT INTO users (name, email, age)
VALUES ('Sara Ali', 'sara@email.com', 28);
```

```
Insert multiple rows at once
INSERT INTO users (name, email, age) VALUES
    ('Ahmed', 'ahmed@email.com', 32),
    ('Layla', 'layla@email.com', 25),
    ('Omar', 'omar@email.com', 29);
```

```
Insert data from another table
INSERT INTO users_backup (name, email)
SELECT name, email FROM users WHERE active = 1;
```

Best Practice

Always specify column names in INSERT statements. If the table structure changes later (new columns added), your inserts won't break.

4.3 UPDATE - Modifying Existing Data

UPDATE changes existing data in a table. This is used when a user edits their profile, changes their password, or you need to fix a value in the database.

```
Update one column for a specific user
UPDATE users SET name = 'Sara Ahmed' WHERE id = 1;

Update multiple columns at once
UPDATE users
SET name = 'Sara Ahmed', email = 'sara.ahmed@email.com'
WHERE id = 1;

Update all rows meeting a condition
UPDATE users SET active = 0 WHERE last_login < '2023-01-01';
```

Critical Warning

Always use a WHERE clause with UPDATE. Without it, you will update EVERY row in the table.

4.4 DELETE - Removing Data

DELETE removes specific rows from a table. The table itself stays intact — only the matching rows are removed.

```
Delete a specific user
DELETE FROM users WHERE id = 1;

Delete multiple rows matching a condition
DELETE FROM users WHERE active = 0 AND created_at < '2022-01-01';

Delete all rows (dangerous! use TRUNCATE instead if intentional)
DELETE FROM users;
```

	DELETE	TRUNCATE
Removes specific rows?	Yes (with WHERE)	No - removes ALL rows
Can filter with WHERE?	Yes	No
Can be rolled back?	Yes (in a transaction)	No (usually)
Speed	Slower for large tables	Faster
Triggers fire?	Yes	No

Chapter 5: JOINS - Connecting Tables

One of the most powerful features of relational databases is the ability to combine data from multiple tables using JOINS. This is how you fetch related data without duplicating it across tables.

5.1 Why JOINS Exist

Imagine you have two tables: users and orders. Instead of storing the user's name in every order row, you store the user's ID. Then, when you need both, you JOIN the tables.

```
Sample data setup
CREATE TABLE users (
  id    INT PRIMARY KEY,
  name  VARCHAR(100)
);

CREATE TABLE orders (
  id      INT PRIMARY KEY,
  user_id INT,
  product VARCHAR(100),
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

5.2 Types of JOINS

JOIN Type	Returns	When to Use
INNER JOIN	Only rows that match in BOTH tables	Get orders with their user info
LEFT JOIN	All rows from left table + matching from right	Get all users, even those with no orders
RIGHT JOIN	All rows from right table + matching from left	Get all orders, even with missing user data
FULL OUTER JOIN	All rows from both tables, matched where possible	Complete picture of both tables
CROSS JOIN	Every combination of rows from both tables	Generating all possibilities
SELF JOIN	Joins a table with itself	Employee-manager hierarchy

5.3 JOIN Examples

INNER JOIN - Only Matched Data

```
**Get all orders with the user's name  
SELECT users.name, orders.product  
FROM users  
INNER JOIN orders ON users.id = orders.user_id;
```

Only users who HAVE orders will appear

LEFT JOIN - All Users, Even Without Orders

```
SELECT users.name, orders.product  
FROM users  
LEFT JOIN orders ON users.id = orders.user_id;
```

All users appear. Users without orders show NULL in product column.

Multiple JOINS

```
**Three tables: users, orders, products  
SELECT  
    users.name      AS customer,  
    products.name   AS product,  
    orders.quantity,  
    orders.total  
FROM orders  
INNER JOIN users    ON orders.user_id    = users.id  
INNER JOIN products ON orders.product_id = products.id  
WHERE orders.total > 100  
ORDER BY orders.total DESC;
```

Chapter 6: Aggregate Functions & Grouping

Aggregate functions perform calculations on groups of rows and return a single value. They are used with SELECT to summarize data.

Function	What It Does	Example
COUNT()	Counts the number of rows	COUNT(*) — total rows
SUM()	Adds up numeric values	SUM(price) — total revenue
AVG()	Calculates the average	AVG(age) — average age
MIN()	Finds the smallest value	MIN(price) — cheapest product
MAX()	Finds the largest value	MAX(score) — highest score

6.1 Using GROUP BY

GROUP BY groups rows by a column so aggregate functions apply per group, not the whole table.

```
**Count how many users are in each city
SELECT city, COUNT(*) AS total_users
FROM users
GROUP BY city;

**Total revenue per product category
SELECT category, SUM(price) AS revenue
FROM orders
GROUP BY category
ORDER BY revenue DESC;
```

6.2 HAVING - Filtering Groups

HAVING is like WHERE, but it filters after grouping. You use it when you want to filter based on the result of an aggregate function.

```
**Get cities that have more than 100 users
SELECT city, COUNT(*) AS total
FROM users
GROUP BY city
HAVING COUNT(*) > 100;

- The difference:
- WHERE filters BEFORE grouping (filters individual rows)
- HAVING filters AFTER grouping (filters groups)
```

Chapter 7: Quick Reference Cheat Sheet

DDL Commands

```
CREATE TABLE table_name (column definitions);
ALTER TABLE table_name ADD COLUMN col_name datatype;
ALTER TABLE table_name DROP COLUMN col_name;
ALTER TABLE table_name MODIFY COLUMN col_name new_datatype;
DROP TABLE table_name;
DROP TABLE IF EXISTS table_name;
TRUNCATE TABLE table_name;
```

DML Commands

```
**SELECT
SELECT col1, col2 FROM table WHERE condition ORDER BY col LIMIT n;

**INSERT
INSERT INTO table (col1, col2) VALUES (val1, val2);

**UPDATE
UPDATE table SET col1 = val1 WHERE condition;

**DELETE
DELETE FROM table WHERE condition;
```

Common Data Types

Data Type	Stores	Example Use
INT	Whole numbers	id, age, quantity
VARCHAR(n)	Text up to n characters	name, email
TEXT	Large blocks of text	descriptions, comments
DECIMAL(p,s)	Precise decimal numbers	price (DECIMAL(10,2))
BOOLEAN	True or false (1/0)	is_active, is_verified
DATE	Date only (YYYY-MM-DD)	birth_date
DATETIME	Date and time	created_at, updated_at
TIMESTAMP	Date/time auto-tracked	last_login

The One-Line Difference

Always Remember

DDL = Working on the TABLE ITSELF (structure, blueprint, schema) DML = Working on the DATA
INSIDE the table (rows, records, values)